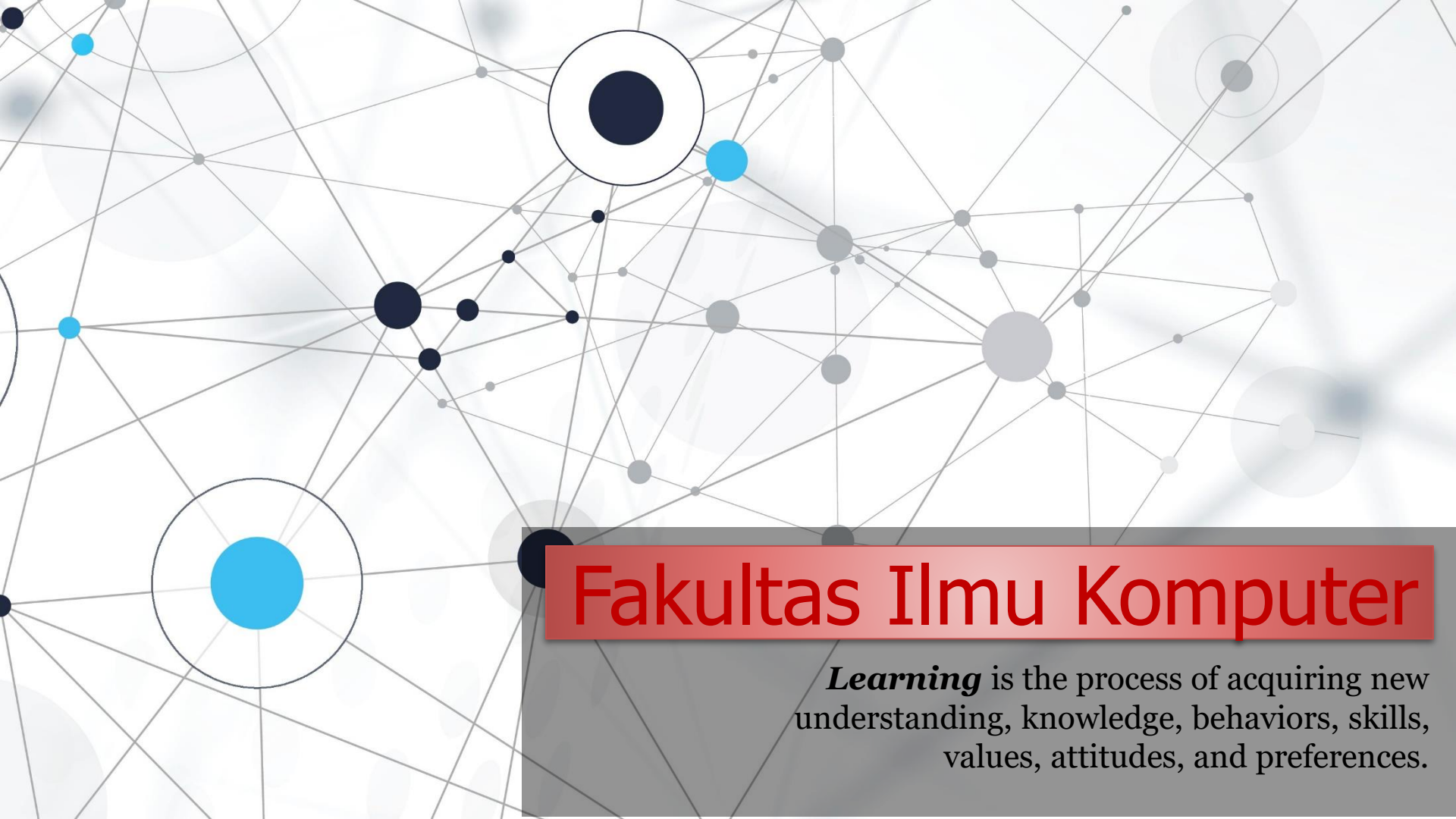


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Python OOP - Method Overloading



Method Overloading Concepts

```
class Perpustakaan:  
    def cari_buku(self, judul):  
        # mencari buku berdasarkan judul  
        pass  
  
    def cari_buku(self, judul, penulis):  
        # mencari buku berdasarkan judul dan penulis  
        pass  
  
    def cari_buku(self, isbn):  
        # mencari buku berdasarkan nomor ISBN  
        pass
```



Dalam perpustakaan, terdapat berbagai jenis buku dengan jenis, jumlah halaman, dan isi yang berbeda-beda. Namun, ketika kita ingin mencari atau meminjam buku dari perpustakaan, kita dapat menggunakan nama buku atau nama pengarang untuk memudahkan pencarian.

Analoginya !!!,

Dalam method overloading, kita dapat menggunakan nama metode yang sama untuk memudahkan pemanggilan metode, meskipun metode tersebut memiliki parameter yang berbeda.

Method Overloading Concepts

```
class Gitar:
    def mainkan(self, akord):
        print("Memainkan akord", akord)

    def mainkan(self, akord, durasi):
        print("Memainkan akord", akord, "dalam durasi", durasi)

g = Gitar()
g.mainkan("C")
g.mainkan("Am", 3)
```



Alat musik gitar dengan banyak variasi akord dan nada yang berbeda-beda. Meskipun memiliki nama yang sama, seperti akord C misalnya, tetapi terdapat banyak variasi nada dan posisi jari yang berbeda untuk menghasilkan suara yang berbeda pula.

Analoginya, dalam method overloading, metode dengan nama yang sama dapat memiliki berbagai macam parameter yang berbeda untuk menghasilkan keluaran yang berbeda pula.

Method Overloading Concepts

```
class Restoran:  
    def pesan(self, menu):  
        print("Memesan menu", menu)  
  
    def pesan(self, menu, tingkat_pedas):  
        print("Memesan menu", menu, "dengan tingkat pedas", tingkat_pedas)  
  
r = Restoran()  
r.pesan("Nasi Goreng")  
r.pesan("Ayam Bakar", "Pedas")
```



Di dalam sebuah Restoran, pasti memiliki menu restoran yang memiliki variasi bahan dan cara penyajian yang berbeda-beda. Meskipun memiliki nama yang sama, seperti nasi goreng misalnya, tetapi terdapat variasi bahan dan cara penyajian yang berbeda-beda untuk menghasilkan rasa yang berbeda pula.

Analoginya, dalam method overloading, metode dengan nama yang sama dapat memiliki berbagai macam parameter yang berbeda untuk menghasilkan keluaran yang berbeda pula.

A decorative border of green leaves and branches at the top of the slide.

1 – Method Overloading *in* Java

A decorative border of green leaves and branches at the bottom of the slide.

What is Method Overloading?

```
Java program to demonstrate working of method
// overloading in Java

public class Sum {

    // Overloaded sum(). This sum takes two int parameters
    public int sum(int x, int y) { return (x + y); }

    // Overloaded sum(). This sum takes three int parameters
    public int sum(int x, int y, int z)
    {
        return (x + y + z);
    }

    // Overloaded sum(). This sum takes two double
    // parameters
    public double sum(double x, double y)
    {
        return (x + y);
    }

    // Driver code
    public static void main(String args[])
    {
        Sum s = new Sum();
        System.out.println(s.sum(10, 20));
        System.out.println(s.sum(10, 20, 30));
        System.out.println(s.sum(10.5, 20.5));
    }
}
```

- ❖ Method Overloading allows different **methods to have the same name**, but different **signatures** where the signature can differ by the **number of input parameters** or **type of input parameters**, or a mixture of both.
- ❖ **Method overloading** is also known as Compile-time Polymorphism, Static Polymorphism, or Early binding in Java.
- ❖ In Method overloading compared to parent argument, child argument will get the highest priority.

Different Ways of Method Overloading in Java

- 1) Changing the Number of Parameters.
- 2) Changing Data Types of the Arguments.
- 3) Changing the Order of the Parameters of Methods

What is Method Overloading?

```
Java program to demonstrate working of method
// overloading in Java

public class Sum {

    // Overloaded sum(). This sum takes two int parameters
    public int sum(int x, int y) { return (x + y); }

    // Overloaded sum(). This sum takes three int parameters
    public int sum(int x, int y, int z)
    {
        return (x + y + z);
    }

    // Overloaded sum(). This sum takes two double
    // parameters
    public double sum(double x, double y)
    {
        return (x + y);
    }

    // Driver code
    public static void main(String args[])
    {
        Sum s = new Sum();
        System.out.println(s.sum(10, 20));
        System.out.println(s.sum(10, 20, 30));
        System.out.println(s.sum(10.5, 20.5));
    }
}
```

- ❖ Method overloading adalah kemampuan untuk mendefinisikan beberapa **metode dengan nama yang sama** di dalam sebuah kelas, tetapi dengan **parameter yang berbeda**.
- ❖ Dalam method overloading, terdapat beberapa **metode yang memiliki nama yang sama** tetapi memiliki jumlah, tipe, atau urutan parameter yang berbeda.
- ❖ Saat memanggil metode yang di-overload, pemilihan metode yang akan dipanggil akan **bergantung pada jumlah dan tipe parameter** yang diberikan pada saat pemanggilan metode.

Method Overloading Example

Without Method Overloading

```
int add2(int x, int y)
{
    return(x+y);
}
int add3(int x, int y, int z)
{
    return(x+y+z);
}
int add4(int w, int x, int y, int z)
{
    return(w+x+y+z);
}
```

With Method Overloading

```
int add(int x, int y)
{
    return(x+y);
}
int add(int x, int y, int z)
{
    return(x+y+z);
}
int add(int w, int x, int y, int z)
{
    return(w+x+y+z);
}
```

Dalam Overloading, **fungsi yang dipanggil akan bergantung pada parameter yang diberikan saat pemanggilan fungsi**, sementara dalam Overriding, metode yang dipanggil akan bergantung pada tipe objek yang dipanggil.

Method Overloading Example

```
float add(int a, float b);
```

```
float add(int a, float b, int c);
```

```
float add(float a, int b);
```

```
float add(float a, float b);
```

Valid Overloading signatures

```
int abc(int num);
```

```
float abc(int num);
```

Invalid Overloading signatures

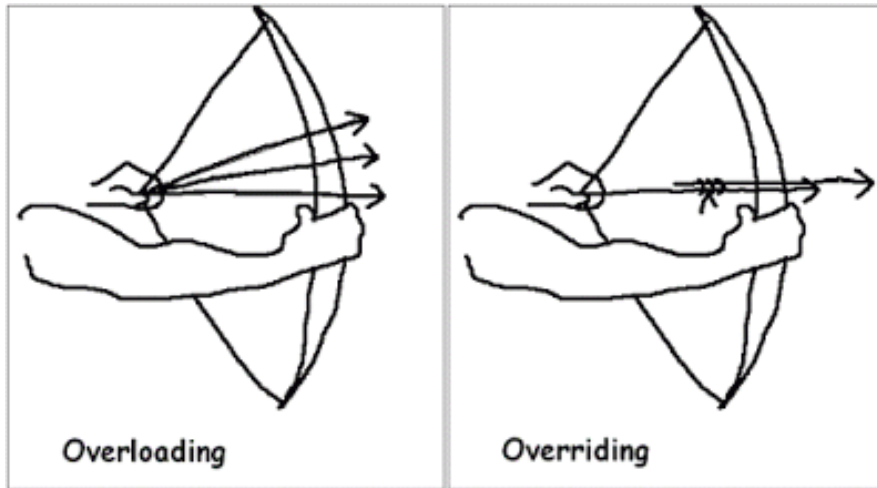
```
class Cat{  
  
    public void Sound(){  
        System.out.println("meow");  
    }  
  
    //overloading method  
    public void Sound(int num){  
        for(int i=0; i<2;i++){  
            System.out.println("meow");  
        }  
    }  
}
```

OVERLOADING

**Same method
name but
different
parameters**

Overloading vs Overriding

Overloading dan Overriding adalah dua konsep yang berbeda dalam pemrograman berorientasi objek (OOP).



What is the difference between Overloading and Overriding?

- 1) **Overloading** is about the same function have different signatures.
- 2) **Overriding** is about the same function, same signature but different classes connected through inheritance.

- 1) **Overloading** adalah kemampuan untuk mendefinisikan beberapa fungsi atau metode dengan nama yang sama di dalam sebuah kelas, tetapi dengan parameter yang berbeda. Dengan kata lain, dalam Overloading, terdapat beberapa fungsi yang memiliki nama yang sama tetapi memiliki parameter yang berbeda. Pemilihan fungsi yang akan dipanggil akan bergantung pada jumlah dan tipe parameter yang diberikan pada saat pemanggilan fungsi.
- 2) **Overriding** adalah kemampuan untuk menimpa (*override*) sebuah metode dari superclass oleh subclass. Dalam Overriding, sebuah metode dalam subclass memiliki nama yang sama dengan metode dalam superclass, dengan implementasi yang berbeda. Metode yang akan dipanggil akan bergantung pada objek yang dipanggil dan tipe objek tersebut (*kelas induk atau turunan*).

Overloading vs Overriding

```
class Calculator:
    def add(self, x, y):
        return x + y

    def add(self, x, y, z):
        return x + y + z

obj = Calculator()
print(obj.add(2, 3)) #akan memanggil fungsi pertama dengan 2 parameter
print(obj.add(2, 3, 4)) #akan memanggil fungsi kedua dengan 3 parameter
```

Dengan kata lain, dalam **Overloading**, terdapat beberapa fungsi yang memiliki nama yang sama tetapi memiliki parameter yang berbeda.

Dalam **Overriding**, sebuah metode dalam subclass memiliki nama yang sama dengan metode dalam superclass, dengan implementasi yang berbeda.

```
class Animal:
    def move(self):
        print("Animal is moving")

class Dog(Animal):
    def move(self):
        print("Dog is walking")

dog = Dog()
dog.move() #akan memanggil metode move dalam kelas Dog, bukan Animal
```

Manfaat Method Overloading

[1] Making code easier to read and understand

By using method overloading, we can give the same method name for different operations, so the code becomes easier to read and understand by programmers working on the same project.

[2] Speed up the development process

By using method overloading, we can reuse the same method name for different operations, so we don't need to create a new method every time we want to perform a different operation. This can speed up the development process, as we can focus on implementing new features that require new methods.

[3] Reduce code redundancy

By using method overloading, we can reduce code redundancy, because we don't need to create a new method every time we want to perform a similar operation, but with different input parameters.

[4] Allows flexibility in method invocation

By using method overloading, we can call methods with various input parameters, depending on our needs. This can provide flexibility to the code we create, so that it can be used for a variety of use cases.

Kekurangan Method Overloading

[1] Confusing for new programmers

Excessive use of method overloading **can make the code complicated and difficult for new programmers to understand**. This is because programmers must properly understand how method overloading works and distinguish between method overloading and method overriding.

[2] Potential programming errors

Excessive use of method overloading can also increase the potential for programming errors, especially if programmers do not properly understand how method overloading works. This **can cause bugs in the program** that are difficult to identify and fix.

[3] Requires more computer resources

The use of method overloading **can also require more computer resources**, especially if there are many method overloads used in the program. This **can affect the performance of the program and require more computer resources** to run the program properly.

[4] Increases code complexity

Excessive use of method overloading **can also increase code complexity**, especially if there are many method overloads used in the program. This can make the code difficult to understand and difficult to manage and fix in the future.

A decorative border of green leaves and branches at the top of the slide.

2 - In Python, Method
Overloading *is not* POSSIBLE?

A decorative border of green leaves and branches at the bottom of the slide.

Pemrograman Python & Ruby



Dalam beberapa bahasa pemrograman seperti Java dan C++, method overloading dapat dilakukan dengan mendefinisikan beberapa metode dengan nama yang sama di dalam sebuah kelas, tetapi dengan parameter yang berbeda. Sedangkan dalam **bahasa pemrograman seperti Python dan Ruby, method overloading tidak didukung secara langsung.**

Why Not ???

(1)

Python adalah bahasa pemrograman yang memiliki filosofi "***batteries included***", artinya Python telah menyediakan banyak *built-in library* dan fitur yang mempermudah proses pengembangan. Dalam kasus method overloading, **Python memiliki cara yang lebih elegan untuk menangani masalah tersebut, yaitu dengan menggunakan argumen default dan argumen opsional.**



Why Not ???

(2)

Python menggunakan **tipe data dinamis**, artinya tipe data dari sebuah objek ditentukan secara otomatis pada saat objek dibuat. Hal ini berbeda dengan bahasa pemrograman seperti Java atau C++, yang menggunakan tipe data statis. Karena tipe data di Python ditentukan secara dinamis, maka Python tidak membutuhkan method overloading untuk menangani perbedaan tipe data.



Why Not ???

(3)

Python menerapkan konsep **duck typing**, artinya tipe sebuah objek ditentukan oleh perilaku objek tersebut, bukan oleh jenisnya. Hal ini berbeda dengan bahasa pemrograman seperti Java atau C++, yang menggunakan konsep tipe data statis. Karena Python menggunakan duck typing, maka method overloading tidak diperlukan karena Python hanya peduli dengan perilaku objek, bukan dengan jenisnya.



But We Still Can Perform It !!!

```
def jumlahkan(*args):  
    total = 0  
    for angka in args:  
        total += angka  
    return total  
  
print(jumlahkan(1, 2, 3, 4, 5)) # Output: 15  
print(jumlahkan(10, 20, 30)) # Output: 60  
print(jumlahkan(5)) # Output: 5
```

"*args" berarti "**banyak argumen**" dan ditulis dengan menggunakan tanda asterisk (*). "*args" adalah parameter khusus dalam bahasa pemrograman Python yang digunakan untuk menerima argumen tanpa batas dalam bentuk tuple. Kata kunci "*args" digunakan dalam definisi fungsi untuk menunjukkan bahwa fungsi tersebut menerima argumen dalam jumlah variabel.

- 1) Ketika parameter "*args" digunakan dalam sebuah fungsi, maka fungsi tersebut dapat menerima argumen dalam jumlah variabel. Argumen-argumen tersebut akan dikumpulkan menjadi satu tuple dalam parameter "*args".
- 2) Hal ini memungkinkan kita untuk membuat fungsi yang **dapat menerima argumen dalam jumlah yang bervariasi** tanpa harus menentukan jumlah argumen yang pasti.

Menurut kalian,
Pada saat kalian ~~akan~~ buat ~~kalian~~ punya
project sendiri. Apakah kalian mau
menggunakan/mengimplementasikan
method overloading?



Exercise *for* Students

Exercise #1 [Parameter Special]

```
Spyder (Python 3.9)
File Edit Search Source Run Debug Consoles Projects Tools View Help
... CLASSES\UNKLAB TEACHING\CLASS\Object-Oriented Programming\CODES\Lecture18 [overloading]
example1.py X
1 # create class
2 class Calculator:
3     def __init__(self):
4         print("Simple Calculator !!!");
5
6     # method 1
7     def add(self, x, y):
8         return x + y;
9
10    # method 2
11    def add(self, x, y, z):
12        return x + y + z;
13
14    # method 3
15    def add(self, *args):
16        return sum(args);
17
18    # method 4
19    def sub(self, x, y):
20        return x - y;
21
22    # create object
23    obj = Calculator()
24    print(obj.sub(6, 4)) #akan memanggil fungsi pertama dengan 2 parameter
25    print(obj.add(2, 3)) #akan memanggil fungsi kedua dengan 2 parameter
26    print(obj.add(2, 3, 4)) #akan memanggil fungsi ketiga dengan 3 parameter
27    print(obj.add(2, 3, 4, 5)) #akan memanggil fungsi dengan 4 parameter
28    print(obj.add(2, 3, 4, 5, 6)) #akan memanggil fungsi dengan 5 parameter
29    print(obj.add(2, 3, 4, 5, 6, 7)) #akan memanggil fungsi dengan 6 parameter
30    print(obj.add(2, 3, 4, 5, 6, 7, 8)) #akan memanggil fungsi dengan 7 parameter
31
```

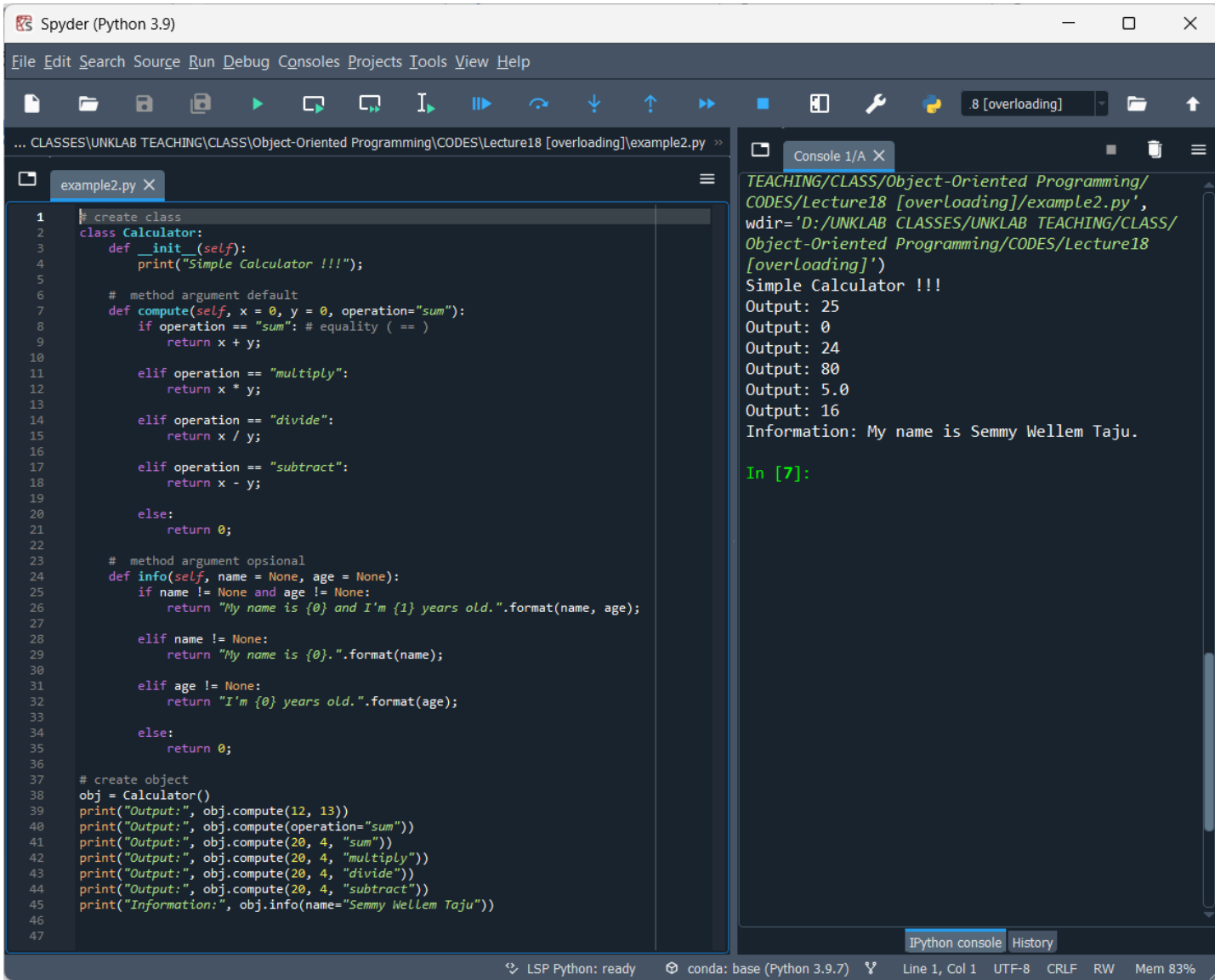
Perhatikan method overloading di samping,

- 1) Apa yang akan terjadi jika tidak ada argument input pada method tersebut? **error** atau **zero**? Mengapa?
- 2) Apa yang akan terjadi jika terdapat 100 input parameter argument pada method tersebut.

```
Lecture18 [overloading]
Simple Calculator !!!
2
5
9
14
20
27
35
In [3]:
```

Menggunakan **args* pada parameter fungsi sehingga bisa menerima jumlah parameter yang berbeda-beda.

Exercise #2 [Parameter Default]



The screenshot displays the Spyder Python IDE interface. The left pane shows a Python script named `example2.py` with the following code:

```
1 # create class
2 class Calculator:
3     def __init__(self):
4         print("Simple Calculator !!!");
5
6     # method argument default
7     def compute(self, x = 0, y = 0, operation="sum"):
8         if operation == "sum": # equality ( == )
9             return x + y;
10
11         elif operation == "multiply":
12             return x * y;
13
14         elif operation == "divide":
15             return x / y;
16
17         elif operation == "subtract":
18             return x - y;
19
20         else:
21             return 0;
22
23     # method argument optional
24     def info(self, name = None, age = None):
25         if name != None and age != None:
26             return "My name is {0} and I'm {1} years old.".format(name, age);
27
28         elif name != None:
29             return "My name is {0}.".format(name);
30
31         elif age != None:
32             return "I'm {0} years old.".format(age);
33
34         else:
35             return 0;
36
37 # create object
38 obj = Calculator()
39 print("Output:", obj.compute(12, 13))
40 print("Output:", obj.compute(operation="sum"))
41 print("Output:", obj.compute(20, 4, "sum"))
42 print("Output:", obj.compute(20, 4, "multiply"))
43 print("Output:", obj.compute(20, 4, "divide"))
44 print("Output:", obj.compute(20, 4, "subtract"))
45 print("Information:", obj.info(name="Semmy Wellem Taju"))
46
47
```

The right pane shows the console output for the script:

```
TEACHING/CLASS/Object-Oriented Programming/
CODES/Lecture18 [overloading]/example2.py',
wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture18
[overloading]')
Simple Calculator !!!
Output: 25
Output: 0
Output: 24
Output: 80
Output: 5.0
Output: 16
Information: My name is Semmy Wellem Taju.

In [7]:
```

The status bar at the bottom indicates: LSP Python: ready, conda: base (Python 3.9.7), Line 1, Col 1, UTF-8, CRLF, RW, Mem 83%.

Question #1

Apa itu method overloading di Python?

- a) Proses menulis dua method yang sama namun dengan tipe parameter yang berbeda.**
- b) Proses menulis dua method yang sama namun dengan jumlah parameter yang berbeda.**
- c) Proses menulis dua method yang sama namun dengan nama yang berbeda.**
- d) Proses menulis dua method yang sama namun dengan nilai parameter yang berbeda.**
- e) Proses menulis dua method yang sama namun dengan tipe return yang berbeda.**

Question #2

Apa yang terjadi jika kita mencoba membuat dua method dengan nama yang sama dan tipe parameter yang sama?

- a) Kode tidak akan kompilasi dan akan muncul error.**
- b) Python akan mengeksekusi method yang pertama ditulis.**
- c) Python akan mengeksekusi method yang terakhir ditulis.**
- d) Python akan meminta Anda untuk menamai salah satu method tersebut.**
- e) Python akan menampilkan pesan error saat runtime.**

Question #3

Apa perbedaan antara method overloading dan method overriding di Python?

- A. Method overloading memungkinkan Anda menulis method dengan nama yang sama, sedangkan method overriding memungkinkan kita menerima method yang ada.
- B. Method overloading memungkinkan Anda menerima method yang ada, sedangkan method overriding memungkinkan Anda menulis method dengan nama yang sama.
- C. Method overloading memungkinkan Anda menulis dua method dengan nama yang sama, sedangkan method overriding memungkinkan Anda menerima method yang ada.
- D. Method overloading hanya bisa dilakukan pada class-class yang berbeda, sedangkan method overriding hanya bisa dilakukan pada class-class yang sama.
- E. Tidak ada perbedaan antara method overloading dan method overriding di Python.

Question #4

Apa yang dimaksud dengan default argument pada method overloading di Python?

- A. Parameter yang memiliki nilai bawaan yang digunakan jika parameter tidak diberikan saat pemanggilan method.
- B. Tipe data parameter yang harus diikuti saat memanggil method.
- C. Jumlah parameter yang harus diikuti saat memanggil method.
- D. Parameter yang harus diberikan saat memanggil method.
- E. Parameter yang tidak dapat diubah saat pemanggilan method.

Question #5

Bagaimana cara menuliskan method overloading di Python?

- A. Dengan menuliskan dua method dengan nama yang sama dan tipe parameter yang sama.
- B. Dengan menuliskan dua method dengan nama yang sama dan jumlah parameter yang sama.
- C. Dengan menuliskan dua method dengan nama yang sama dan tipe return yang sama.
- D. Dengan menuliskan dua method dengan nama yang sama namun dengan parameter yang berbeda.
- E. Tidak ada cara khusus untuk menuliskan method overloading di Python.

Question #6

Apa itu default parameter pada method overloading di Python?

- A. Default parameter adalah parameter yang harus diisi nilai saat pemanggilan method.
- B. Default parameter adalah parameter yang hanya boleh diisi nilai pada saat deklarasi method.
- C. Default parameter adalah parameter yang diisi nilai secara otomatis pada saat pemanggilan method.
- D. Default parameter adalah parameter yang diisi nilai secara otomatis pada saat deklarasi method.
- E. Default parameter adalah parameter yang diisi secara acak pada saat pemanggilan method.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

